

Introduction to Decision-Focused Learning for airlines

Part 2: Decision-Focused Learning

Léo Baty

22/09/2026



ÉCOLE NATIONALE DES
PONTS
ET CHAUSSÉES



IP PARIS

AIRFRANCE 

Part 2: outline

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

Summary of part 1

Part 1

- The **stochastic tail assignment problem**
- Solved by **column generation** with a **diving heuristic**
- **Near-optimal** decisions (gap of 0.3% at 100 scenarios) but too **slow**

Part 2

- Learn a **policy** mapping an instance and its features to **arc costs**
- Feed them to a **fast deterministic solver**
- Reproduce Part 1 decisions at **operational speed**

The framework that does this is called **Decision-Focused Learning**.

Decision-Focused Learning policies

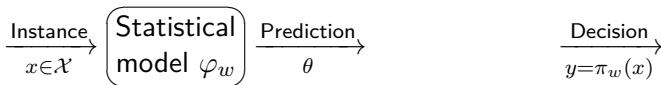
- **Learning problem:** find π such that $\pi(x)$ is “good”.

Instance
 $\xrightarrow{x \in \mathcal{X}}$

Decision
 $\xrightarrow{y = \pi(x)}$

Decision-Focused Learning policies

- **Learning problem:** find w such that $\pi_w(x)$ is “good”.

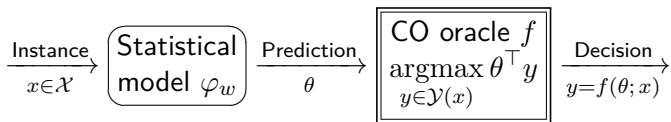


What is needed to build a DFL policy?

1. A **statistical model** φ_w (GLM, NN, GNN, ...).

Decision-Focused Learning policies

- **Learning problem:** find w such that $\pi_w(x) = f(\varphi_w(x); x)$ is “good”.

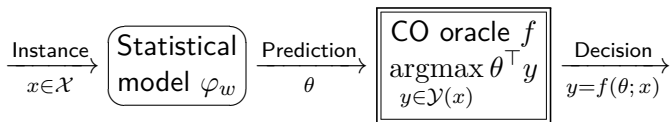


What is needed to build a DFL policy?

1. A **statistical model** φ_w (GLM, NN, GNN, ...).
2. A **CO oracle** f .

Decision-Focused Learning policies

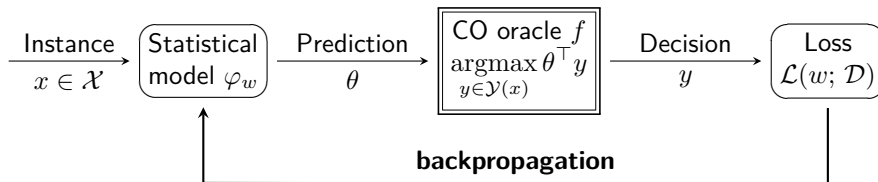
- **Learning problem:** find w such that $\pi_w(x) = f(\varphi_w(x); x)$ is “good”.



What is needed to build a DFL policy?

1. A **statistical model** φ_w (GLM, NN, GNN, ...).
2. A **CO oracle** f .
3. A **dataset** $\mathcal{D}$.
4. A **learning algorithm**.

Training the policy



Supervised learning

Dataset: $\mathcal{D} = (x_i, \bar{y}_i)_{i \in [N]}$

Learning problem:

$$\min_w \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\varphi_w(x_i)), \bar{y}_i)$$

Empirical Risk Minimization

Dataset: $\mathcal{D} = (x_i)_{i \in [N]}$

Learning problem:

$$\min_w \frac{1}{N} \sum_{i=1}^N c(f(\varphi_w(x_i)))$$

Why it is hard

The CO oracle $f: \theta \mapsto \operatorname{argmax}_{y \in \mathcal{Y}(x)} \theta^\top y$ is **piecewise constant** in θ :

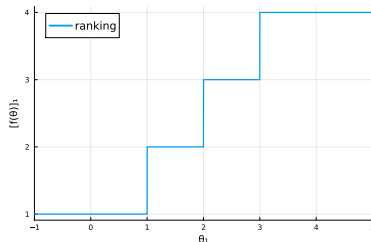
- tiny changes in $\theta \rightarrow$ same decision $\rightarrow$ gradient = 0,
- at the boundaries $\rightarrow$ the decision jumps $\rightarrow$ gradient undefined.

No usable gradient $\implies$ no end-to-end training.

$$\text{ranking: } \theta \mapsto \operatorname{argmax}_{y \in \sigma(n)} \theta^\top y,$$

$\sigma(n)$: permutations of $\{1, \dots, n\}$ (ranking vectors)

$$\theta_1 \mapsto [\text{ranking}([\theta_1, 3, 1, 2])]_1$$



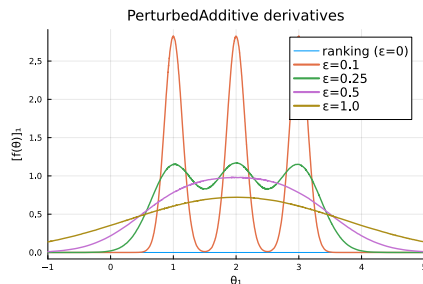
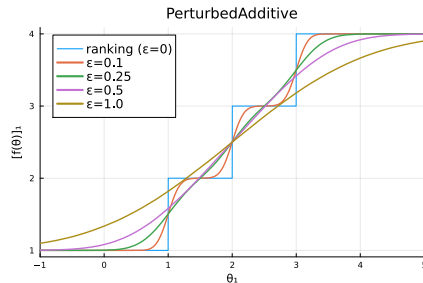
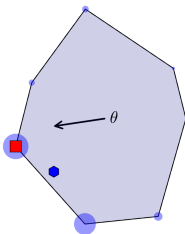
- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

Smoothing by perturbation

$$f(\theta) = \operatorname{argmax}_{y \in \mathcal{Y}(x)} \theta^\top y$$

$$\hat{f}(\theta) = \mathbb{E}_Z \left[\operatorname{argmax}_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right]$$

$$\varepsilon \in \mathbb{R}_+, Z \sim \mathcal{N}(0, I)$$



Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where Ω is a convex regularization function.

The perturbed $\hat{f}$ from the previous slide is a regularized argmax for this Ω : this is what makes the Fenchel-Young machinery apply.

Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where Ω is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

The perturbed $\hat{f}$ from the previous slide is a regularized argmax for this Ω : this is what makes the Fenchel-Young machinery apply.

Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where Ω is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

The perturbed $\hat{f}$ from the previous slide is a regularized argmax for this Ω : this is what makes the Fenchel-Young machinery apply.

Perturbed case notation:

$$\mathcal{L}_\Omega = \mathcal{L}_\varepsilon^{FYL}.$$

Non-optimality of $\bar{y}$
as a solution of the
regularized prediction problem

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \max_{y \in \operatorname{conv}(\mathcal{Y}(x))} (\theta^\top y - \Omega(y)) - (\theta^\top \bar{y} - \Omega(\bar{y}))$$

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \Omega^*(\theta) + \Omega(\bar{y}) - \theta^\top \bar{y}$$

Supervised learning: Fenchel-Young losses [Blondel et al. 2020]

$$\hat{f}(\theta) = \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y}(x))} \theta^\top y - \Omega(y)$$

where Ω is a convex regularization function.

Perturbation case:

$$\Omega^*(\theta) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right].$$

Non-optimality of $\bar{y}$
as a solution of the
regularized prediction problem

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \max_{y \in \operatorname{conv}(\mathcal{Y}(x))} (\theta^\top y - \Omega(y)) - (\theta^\top \bar{y} - \Omega(\bar{y}))$$

$$\mathcal{L}_\Omega(\theta; \bar{y}) = \Omega^*(\theta) + \Omega(\bar{y}) - \theta^\top \bar{y}$$

The perturbed $\hat{f}$ from the previous slide is a regularized argmax for this Ω : this is what makes the Fenchel-Young machinery apply.

Perturbed case notation:

$$\mathcal{L}_\Omega = \mathcal{L}_\varepsilon^{FYL}.$$

Properties that make SGD tractable

- $\mathcal{L}_\Omega(\theta; \bar{y}) \geq 0$
- $\mathcal{L}_\Omega(\theta; \bar{y}) = 0 \Leftrightarrow \bar{y} = \nabla \Omega^*(\theta)$
- Convex in θ
- $\nabla_\theta \mathcal{L}_\Omega(\theta; \bar{y}) = \hat{f}_\Omega(\theta) - \bar{y}$

Building a differentiable loss

Generalization for $f: \theta \mapsto \underset{y}{\operatorname{argmax}} \theta^\top g(y) + h(y)$. Setting $g(y) = y$ and $h = 0$ recovers the Fenchel-Young loss $\mathcal{L}_\Omega$ from the previous slide, with Ω induced by the perturbation.

Theorem [Berthet et al. 2020, generalization to non-linear costs by Baty, Jungel, Klein, Parmentier, Schiffer, Transportation Science 2024]

The perturbed loss function

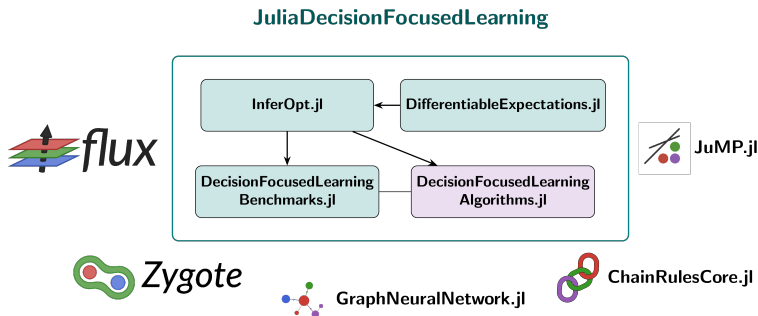
$$\mathcal{L}_\varepsilon(\theta, \bar{y}) = \mathbb{E} \left[\max_{y \in \mathcal{Y}} (\theta + \varepsilon Z)^\top g(y) + h(y) \right] - (\theta^\top g(\bar{y}) + h(\bar{y}))$$

with $\varepsilon \in \mathbb{R}_+$, and $Z \sim \mathcal{N}(0, I_d)$, is convex and differentiable in θ

$$\begin{aligned} \nabla_\theta \mathcal{L}_\varepsilon(\theta, \bar{y}) &= \mathbb{E} \left[g \left(\underset{y \in \mathcal{Y}}{\operatorname{argmax}} (\theta + \varepsilon Z)^\top g(y) + h(y) \right) \right] - g(\bar{y}) \\ &= \mathbb{E}[g(f(\theta + \varepsilon Z))] - g(\bar{y}) \end{aligned}$$

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

Three main packages



- | | |
|-------------------------------|--|
| <code>InferOpt.jl</code> | differentiable CO layers + losses (core building blocks) |
| <code>DFLBenchmarks.jl</code> | standardized problems behind one interface |
| <code>DFLAlgorithms.jl</code> | black-box training algorithms in one line |

A tour of DFLBenchmarks

Three main categories of benchmarks:

Family	Benchmarks
AbstractBenchmark	Argmax, Argmax2D, Ranking, SubsetSelection, PortfolioOptimization, FixedSizeShortestPath, Warcraft
AbstractStochasticBenchmark	StochasticVehicleScheduling, ContextualStochasticArgmax
AbstractDynamicBenchmark	DynamicVehicleScheduling, DynamicAssortment, Maintenance

A tour of DFLBenchmarks

Three main categories of benchmarks:

Family	Benchmarks
AbstractBenchmark	Argmax, Argmax2D, Ranking, SubsetSelection, PortfolioOptimization, FixedSizeShortestPath, Warcraft
AbstractStochasticBenchmark	StochasticVehicleScheduling, ContextualStochasticArgmax
AbstractDynamicBenchmark	DynamicVehicleScheduling, DynamicAssortment, Maintenance

- **Evaluate a new algorithm:** pick a benchmark, train a policy, evaluate
 - ▶ change one line to change the problem
- **Implement a new problem** using the common interface.
 - ▶ gives access to the whole ecosystem features

The interface is designed to be easy to extend.

Using a benchmark

A benchmark hands you the needed **three components** to initialize a DFL policy:

```
bench = StochasticVehicleSchedulingBenchmark() # Choose a benchmark problem
```

```
# 1. Statistical model: a Flux model (features -> values)
```

```
model = generate_statistical_model(bench)
```

```
# 2. CO oracle: the combinatorial maximizer
```

```
maximizer = generate_maximizer(bench)
```

```
# 3. data: a dataset of (features, target) samples
```

```
train_data = generate_dataset(bench, 100; seed=1)
```

```
# 4. apply your learning algorithm here
```

```
# ...
```

Swap `bench` to a new problem, **same three lines**.

A dataset is a vector of DataSample

Each DataSample bundles everything about one instance:

```
struct DataSample
    x          # input features
    y          # target solution (if it applies)
    context    # observed input data (instance, context information, ...)
    extra      # any extra non-solver data: scenario, reward, ...
end
```

Applying the policy to one sample:

```
sample = train_data[1]
 $\theta$  = model(sample.x)          # features    -> prediction
y = maximizer( $\theta$ ; sample.context...) # prediction -> decision
```

context is spread as kwargs into the oracle, **extra** never touches the solver.

Algorithms in DFL Algorithms

The right algorithm depends on the considered **problem**:

	Imitation	ERM
	fit precomputed targets	minimize expected cost
static	Fenchel-Young imitation	Structured RL
stochastic	Fenchel-Young imitation	Mirror Descent, Structured RL
dynamic	Anticipative imitation, DAgger	Structured RL

Training with an algorithm

Bundle the predictor and oracle into a `DFLPolicy`, pick an algorithm, train.

```
# 1. policy = predictor + CO oracle  
policy = DFLPolicy(model, maximizer)
```

```
# 2. a decision-focused training algorithm  
algo = PerturbedFenchelYoungLossImitation(; nb_samples=10,  $\varepsilon$ =1.0)
```

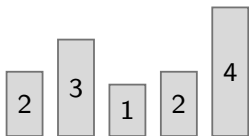
```
# 3. train end to end (metrics logged each epoch)  
history = train_policy!(  
    algo, policy, train_data; epochs=50, metrics=(gap_metric,)  
)
```

Same call, **swap the algorithm**: `MirrorDescent`, `Dagger`, ...

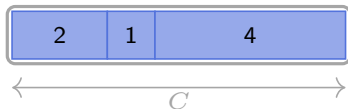
- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

The contextual stochastic knapsack problem

- **Instance:** n items with integer weights c and a capacity C .
- **Context** x : observed features correlated with the item values.
- **Scenario** $\xi \in \mathbb{R}^n$: the **realized item values**, drawn at random and revealed *after* we decide: $\xi = u_{\text{base}} + M x_{\text{raw}} + \text{noise}$, with M unknown.
- **Decision** $y \in \{0, 1\}^n$: which items to take, under $c^\top y \leq C$.

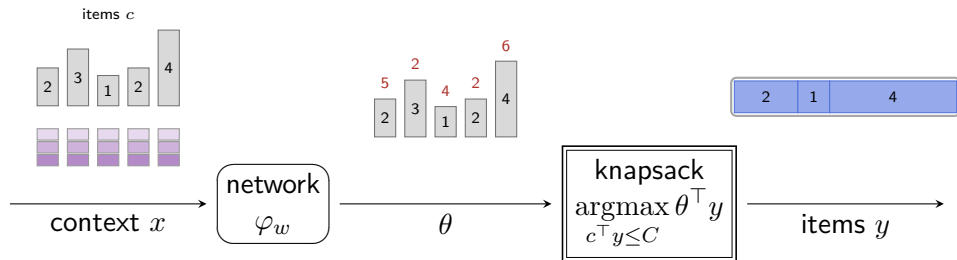


Instance: items with weights c



Decision y : chosen items, $c^\top y \leq C$

The DFL policy



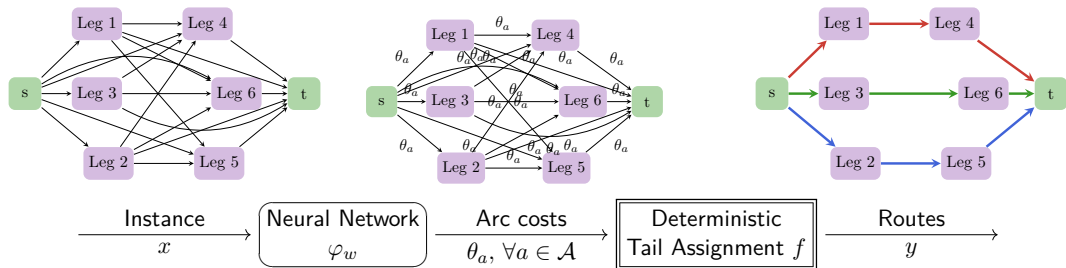
- **Policy** $\pi_w(x) = f(\varphi_w(x))$.
- **Target** $\bar{y}$: the *anticipative* solution (the knapsack solved on the realized values ξ).
- **Learn** the weights w by Fenchel-Young imitation.

Live demo

→ 04_dfl_demo_knapsack.jl

- 1 What is Decision-Focused Learning?
- 2 Learning through an optimization layer
- 3 The JuliaDecisionFocusedLearning ecosystem
- 4 Demo: contextual stochastic knapsack problem
- 5 DFL for stochastic tail assignment
- 6 Practice session 2

Target 1: imitating integer solutions

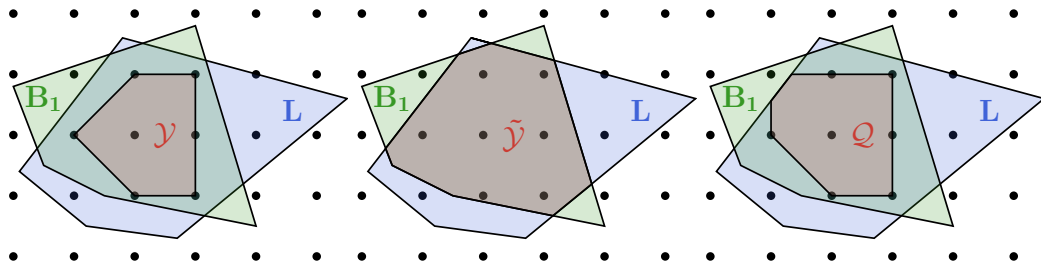


Fenchel-Young loss over integer solutions ($\bar{y} \in \mathcal{Y}(x)$):

$$\mathcal{L}_y^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Y}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y},$$

This loss can no longer be driven to zero when $\bar{y}$ lies outside the convex hull of feasible solutions, so it loses its meaning as a loss on relaxation solutions $\bar{y} \in \tilde{\mathcal{Y}}(x)$.

Target 2: imitating column generation relaxation solutions



- Y : feasible (integer) routes
- $\tilde{Y}$: relaxed integer constraints
- Q : column generation relaxation

For the stochastic tail assignment: $Q = \tilde{Y}$ (flow constraints). The convex hull of source-to-sink paths in a directed acyclic graph is exactly the unit flow polytope, so the Dantzig-Wolfe reformulation does not strengthen the relaxation here.

Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ($\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[\max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ($\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[\max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

During training, we only need to use an LP **relaxation**:

$$\tilde{f}: \theta \longmapsto \operatorname{argmax}_{y \in \tilde{\mathcal{Y}}(x)} \theta^\top y$$

Target 2: imitating column generation relaxation solutions

Fenchel-Young loss over column generation relaxation solution ($\bar{y} \in \mathcal{Q} = \tilde{\mathcal{Y}}$):

$$\mathcal{L}_{\tilde{\mathcal{Y}}}^{\text{FYL}}(\theta, \bar{y}) = \mathbb{E}_Z \left[\max_{y \in \mathcal{Q}} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y} = \mathbb{E}_Z \left[\max_{y \in \tilde{\mathcal{Y}}(x)} (\theta + \varepsilon Z)^\top y \right] - \theta^\top \bar{y}.$$

During training, we only need to use an LP **relaxation**:

$$\tilde{f}: \theta \longmapsto \operatorname{argmax}_{y \in \tilde{\mathcal{Y}}(x)} \theta^\top y$$

Upsides:

- **faster** training
- no need to find good integer solutions, **column generation relaxation is enough**

Experiment setup

Instances:

- Train/validation: small instances
- Test: larger instances

Features: distributional information about connection slacks

Solutions:

- $\mathcal{D}^{\tilde{\mathcal{Y}}}$: dataset with solutions of the column generation relaxation with 100 scenarios
- $\mathcal{D}^{\mathcal{Y}}$: dataset with integer solutions (from diving heuristic) with 100 scenarios

Results (smaller is better)

Evaluation dataset	Train	Val	Test	Diving heuristic
Average run time	31s	31s	46s	2061s

Results (smaller is better)

Evaluation dataset	Train	Val	Test	Diving heuristic
Average run time	31s	31s	46s	2061s

Training dataset	$\mathcal{D}^{\tilde{\mathcal{Y}}}$			$\mathcal{D}^{\mathcal{Y}}$		
Evaluation dataset	Train	Val	Test	Train	Val	Test
Average gap (to diving heuristic)	1.7%	2.2%	2.9%	1.7%	1.5%	2.3%

Orders of magnitude faster at about 2% quality loss compared to the diving heuristic.

Practice session 2

→ 05_dfl_notebook.jl